

Compte rendu Devoir Express, Partie III

API de gestion des étudiants, des cours et des inscriptions (Node.js / Express / SQLite)

Nom, Prénom : Maganga, Emilio

Date : 20 août 2026

Partie 1 : Résumé des deux séances Express (concepts clés appris)

Les deux séances consacrées à Express ont permis de découvrir progressivement l'écosystème Node.js et le framework Express, à travers une série d'exercices guidés :

- Initialisation d'un projet Node.js avec ``npm init``, structuration du ``package.json`` (scripts, dépendances, métadonnées), et distinction entre ``dependencies`` et ``devDependencies``.
- Hot-reloading avec Nodemon, pour accélérer le cycle de développement en redémarrant automatiquement le serveur à chaque modification de fichier.
- Mesure du temps de démarrage avec la librairie ``moment``, illustrant l'usage de librairies utilitaires tierces au sein d'un projet Node.js.
- Persistance simple avec ``node-localstorage``, une première approche de stockage de données côté serveur avant l'introduction d'une vraie base de données.
- Création d'un serveur Express minimal (``app.get``, ``app.listen``), point de départ de toute API REST.
- Architecture en couches (services): séparation de la logique métier (CRUD sur étudiants, cours, inscriptions) dans des classes dédiées, indépendantes du routage HTTP — un principe clé pour garder un code testable et maintenable.
- Construction d'une API REST complète: convention des routes (``GET``, ``POST``, ``PUT``, ``DELETE``), codes de statut HTTP, structuration des réponses JSON.

Ces séances ont posé les bases nécessaires pour aborder la Partie III du devoir, qui demandait de transformer ces exercices guidés en une véritable API REST persistée en base de données.

Partie 2 : Description du projet développé

Objectif

Le projet `emilio-express-courses` est une API REST permettant de gérer l'inscription d'étudiants à des cours dispensés par un établissement scolaire. Il couvre trois entités principales : les étudiants, les cours et les inscriptions (relation many-to-many entre les deux premières).

Architecture

Le projet suit une architecture simple en couches :

server.mjs → point d'entrée, définition des routes Express

db.mjs → connexion et initialisation du schéma SQLite

services/

student-service.mjs → logique métier CRUD des étudiants

course-service.mjs → logique métier CRUD des cours

subscription-service.mjs → logique métier des inscriptions

middlewares/

validation.mjs → schémas de validation Joi + middleware générique

error-handler.mjs → middleware centralisé de gestion des erreurs

Services et endpoints

- StudentService (`/students`) : 5 endpoints (`POST`, `GET` liste, `GET`/:id`, `PUT`/:id`, `DELETE`/:id`), recherche par identifiant numérique ou par matricule.
- CourseService (`/courses`) : 5 endpoints, structure identique, recherche par identifiant ou par code de cours.
- SubscriptionService (`/subscriptions`) : 6 endpoints couvrant l'inscription, la désinscription, et la consultation croisée (inscriptions par cours, étudiants par cours, inscriptions par étudiant, cours par étudiant).

Base de données

Le stockage initial en `node-localstorage` a été entièrement remplacé par SQLite (via les bibliothèques `sqlite` et `sqlite3`), avec trois tables (`students`, `courses`, `subscriptions`) et des clés étrangères (`FOREIGN KEY ... ON DELETE CASCADE`) garantissant l'intégrité référentielle entre étudiants, cours et inscriptions.

Validation et gestion des erreurs

- Les données entrantes sont validées avec Joi (schémas dédiés par entité et par type d'opération, création vs mise à jour partielle).

- Chaque route est enveloppée dans un bloc ``try/catch``, avec délégation au middleware d'erreur centralisé (``error-handler.mjs``) pour les erreurs non prévues.
- Les codes de statut HTTP sont utilisés de façon cohérente : ``201`` (création), ``200`` (succès), ``400`` (requête invalide ou contrainte violée), ``404`` (ressource introuvable), ``500`` (erreur serveur).

Partie 3 : Difficultés rencontrées et solutions apportées

1. Identifiant double (id technique vs identifiant métier)

Les énoncés demandaient de pouvoir rechercher un étudiant ou un cours soit par son ``id`` numérique, soit par un identifiant métier (matricule ou code). La solution retenue consiste à tester si le paramètre reçu est numérique ; si oui, on cherche d'abord par ``id``, sinon (ou en absence de résultat) on retombe sur la recherche par ``number`/`code``.

2. Migration de ``node-localstorage`` vers SQLite

Le passage d'un stockage clé-valeur synchrone à une vraie base relationnelle a nécessité de réécrire l'ensemble des services en fonctions asynchrones (``async/await``), et d'adapter tous les appels dans ``server.mjs`` en conséquence. Cela a aussi imposé de réfléchir au schéma relationnel (table de jointure ``subscriptions`` avec clés étrangères) plutôt qu'à un simple objet JSON.

3. Intégrité référentielle des inscriptions

Il fallait empêcher l'inscription d'un étudiant à un cours inexistant, ou la création de doublons. Cela a été résolu par une vérification explicite de l'existence de l'étudiant et du cours avant insertion, combinée à une contrainte ``UNIQUE (student_id, course_id)`` au niveau de la base pour éviter les inscriptions en double.

4. Uniformisation de la gestion des erreurs

Dupliquer la gestion des erreurs dans chaque route aurait rendu le code difficile à maintenir. La mise en place d'un middleware d'erreur centralisé (``error-handler.mjs``), couplé à des blocs ``try/catch`` légers dans chaque route qui font simplement ``next(error)``, a permis de garder un code propre tout en assurant des réponses HTTP cohérentes.

5. Compilation du module natif SQLite sur Apple Silicon

Lors d'une réinstallation des dépendances, le module natif ``sqlite3`` a été compilé pour une architecture incompatible avec le processeur (erreur ``ERR_DLOPEN_FAILED``, architecture

x86_64 au lieu d'arm64). La commande ``npm rebuild sqlite3`` a permis de recompiler le binaire natif pour la bonne architecture et de résoudre le problème, un rappel utile de l'importance de bien compiler les dépendances natives après changement d'environnement.

Partie 4 : Améliorations effectuées et fonctionnalités bonus

Validation des données, avec messages d'erreur explicites en français retournés au client en cas de requête invalide.

Middleware ``express.json()`` et ``express.urlencoded()`` pour parser automatiquement les corps de requêtes JSON et formulaire.

Middleware de gestion d'erreurs centralisé, garantissant des réponses homogènes sur toute l'API.

Documentation complète de l'API dans ``API.md`` : chaque endpoint est documenté avec sa méthode HTTP, son URL, le format JSON attendu et retourné, les codes de statut possibles, et des exemples ``curl``.

Contraintes d'intégrité en base (clés étrangères avec suppression en cascade, contraintes d'unicité sur le matricule, le code de cours et le couple étudiant/cours) plutôt qu'une simple vérification applicative.

Conclusion

Ce projet a permis de consolider des compétences essentielles en développement backend avec Node.js et Express : structuration d'un projet, conception d'une API REST cohérente, persistance des données en base relationnelle, validation des entrées, et gestion propre des erreurs. Le point clé à retenir est l'intérêt d'une architecture en couches (routes / services / base de données) qui isole la logique métier du framework HTTP, rendant le code plus lisible, plus testable et plus facile à faire évoluer par exemple lors de la migration du stockage local vers SQLite, qui n'a nécessité aucune modification des routes elles-mêmes.

Lien du repository GitHub : ``https://github.com/tooluxarala/emilio-express-courses.git``